リモートリポジトリの操作

慶應義塾大学理工学部物理情報工学科
渡辺

今回の目標

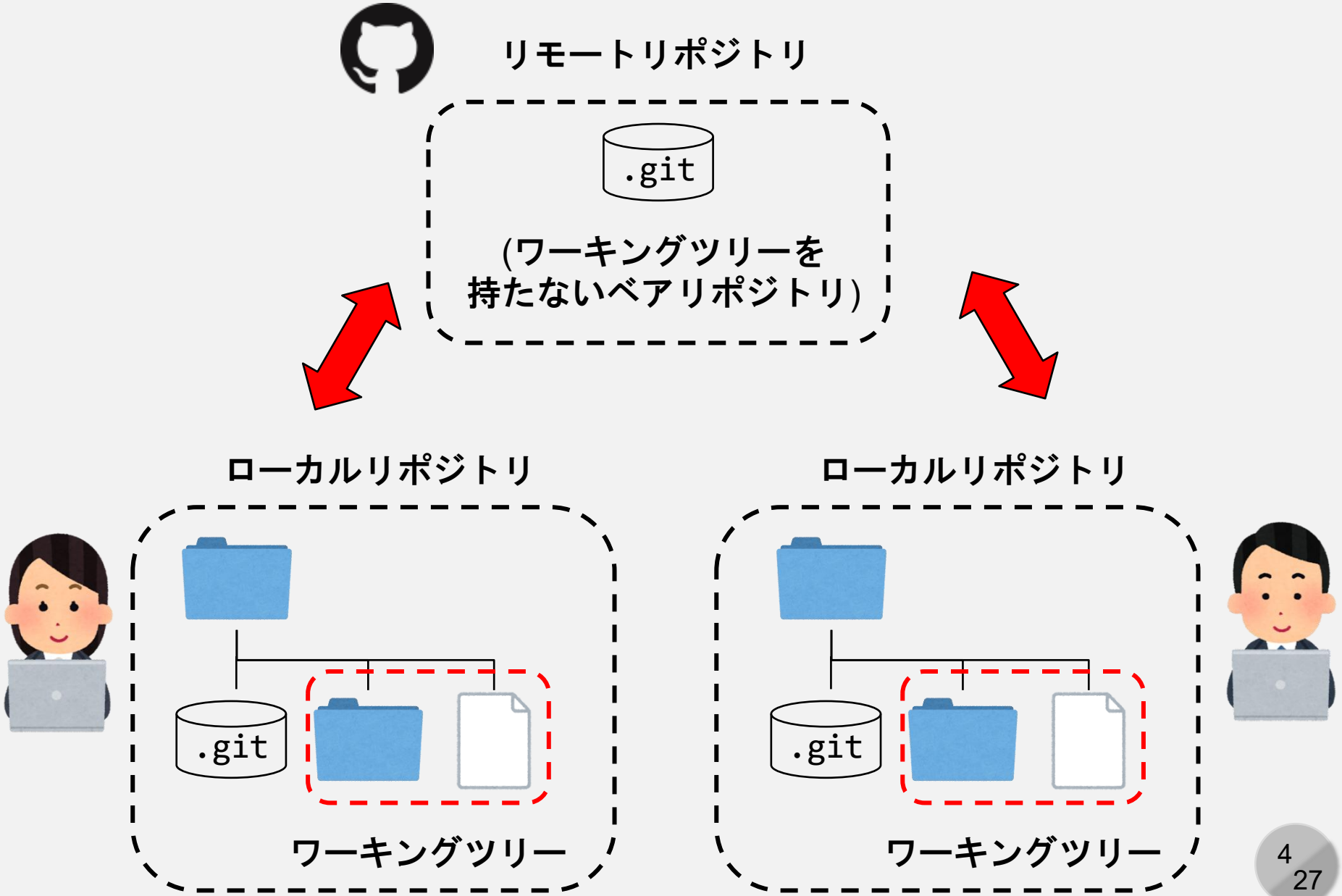
- リモートリポジトリの操作について学ぶ
- 特に、以下について理解する
  - どのコマンドでどのブランチが動くか
  - どのコマンドでどこが同期されるか
  - ローカルとリモートの状態の変化

リモートリポジトリとは

- Gitはローカルに全ての情報を持ち、履歴を管理できる
- しかし、多人数開発の時には、「開発に関わるみんな」が参照可能な場所にリポジトリが欲しい
- 個人開発でも、「どこからでもアクセスできる場所」にリポジトリがあると便利

リモートリポジトリ

リモートリポジトリとは

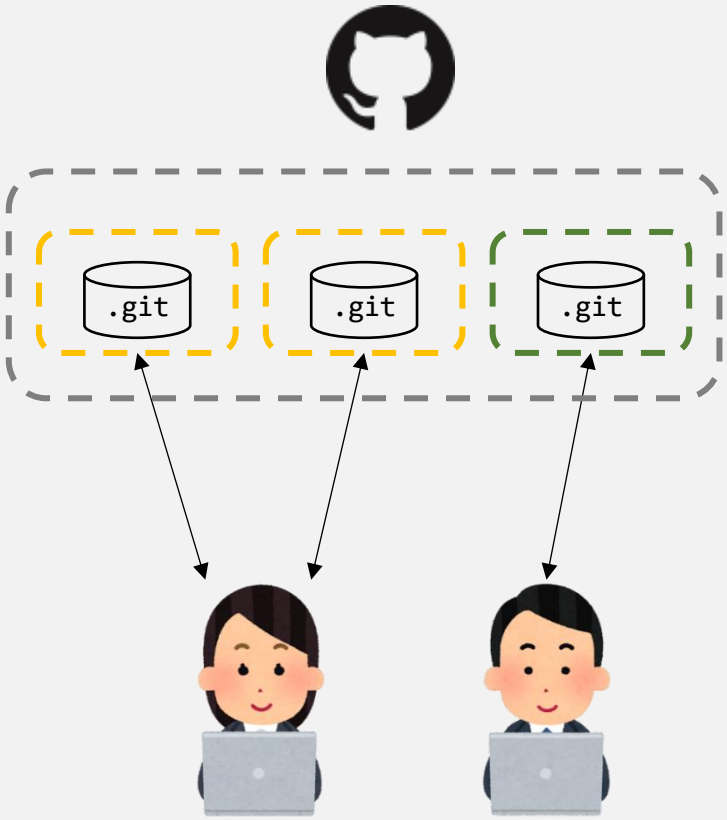


リモートリポジトリとは

リモートリポジトリ

- どこからでもアクセスしたい
- 認証をきっちりしたい
- 自分で作りたくない

➡ ホスティングサービス



ホスティングサービス

企業がサーバを貸し出すサービス
メールやウェブサーバ、データなど
Gitのホスティングサービスを提供するのがGitHub

※ 他にGitLabやBitBucketなど

リモートリポジトリとは

中央リポジトリ

- プロジェクトメンバーで共有するリモートリポジトリ
- 通常、リモートリポジトリはこれ一つ
- Gitは複数のリモートリポジトリに名前をつけて管理できるが、通常はoriginというリモートリポジトリ一つで運用
- Gitホスティングサービスに置くことが多い

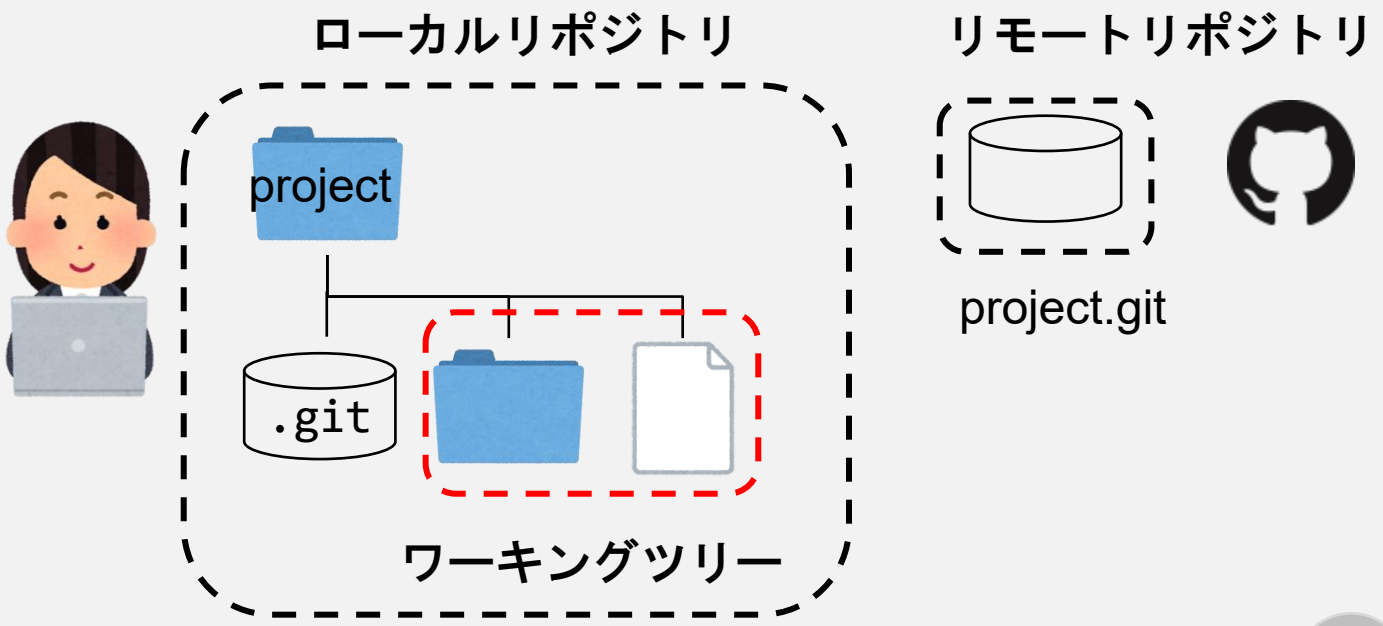
本講義では今後、以下を前提とする

- リモートリポジトリは一つ
- 名前はorigin
- 置き場所はGitHub

ベアリポジトリ

ワーキングツリーを持たないリポジトリのこと
通常のリポジトリの.gitディレクトリがトップレベルディレクトリになったような中身
「プロジェクト名.git」という名前とする

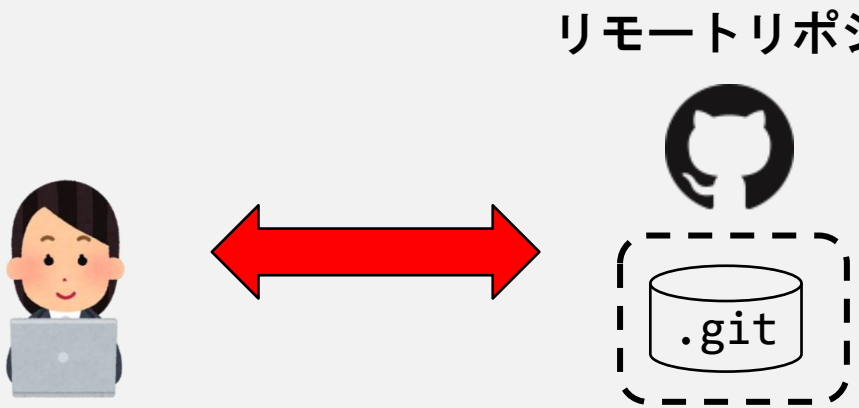
例： projectというプロジェクトなら、リモートリポジトリはproject.gitにする



※ git init --bare で作成可能だが、自分で作ることはほとんどない

認証とプロトコル

リモートリポジトリにアクセスするためには
「場所」「方法」「本人確認」が必要



場所

どこにあるか？

方法

どう通信するか？

本人確認

確かに本人か？

認証とプロトコル

URL (Uniform Resource Locator)

インターネットにおける「住所」
例えば https://www.github.comなど

プロトコル

ここではインターネットにおける通信手段のこと
GitHubへのアクセスは、「SSH」か「HTTPS」

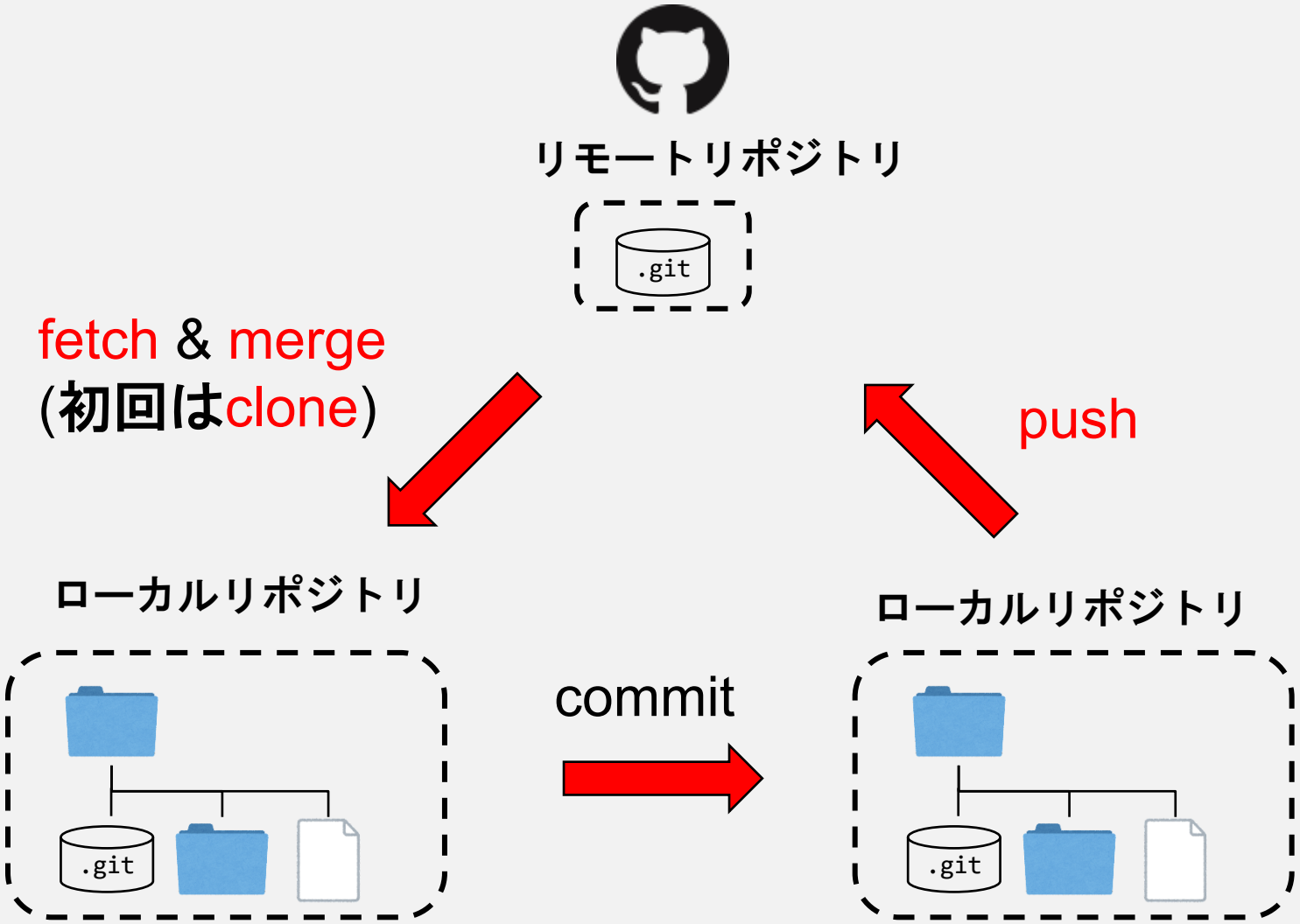
SSH (Secure Shell)

認証方法は「公開鍵認証」←本講義ではこちらを使う

HTTPS (Hypertext Transfer Protocol Secure)

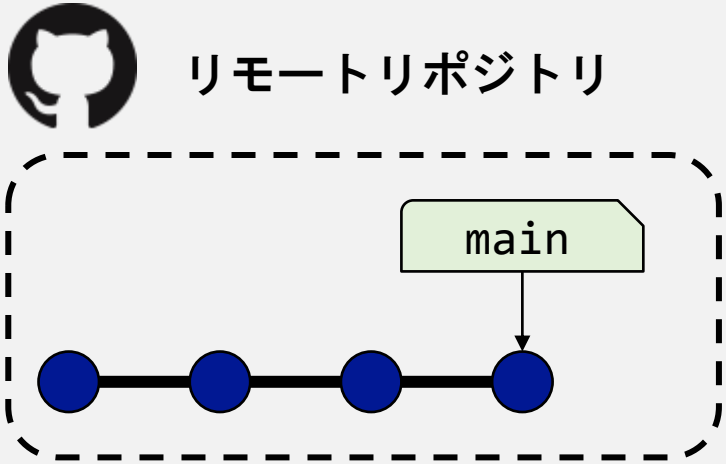
認証方法は「個人アクセストークン」

リモートリポジトリの使い方



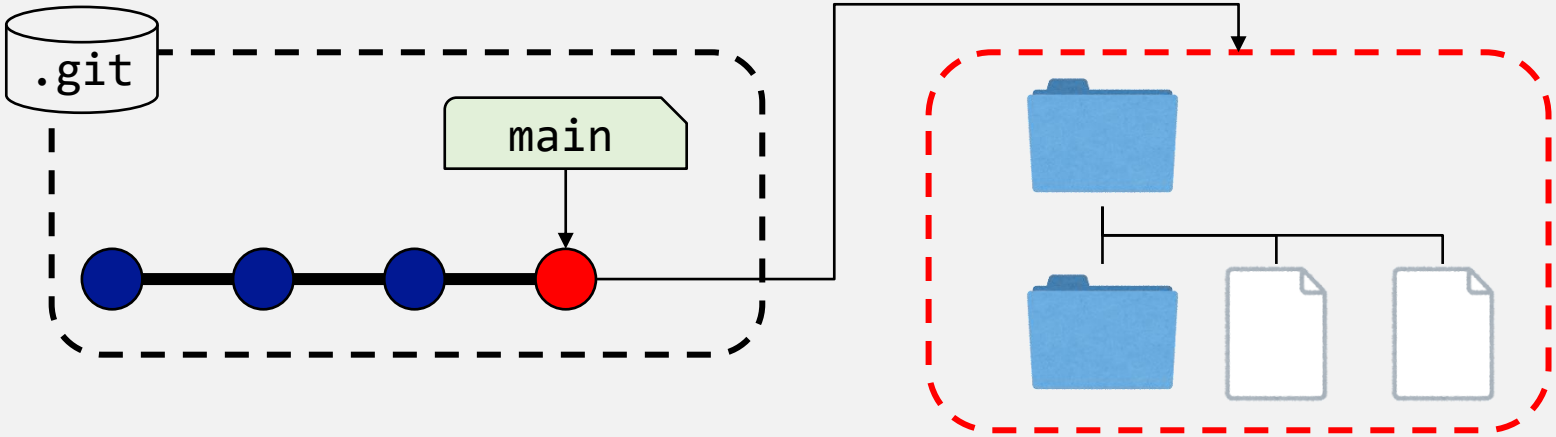
赤字がリモートとやりとりするコマンド

クローン



リモートの「歴史」を持ってきて
最新のワーキングツリーを展開する

```
$ git clone URL/to/remote.git
```



クローン

```
$ git clone リモートリポジトリのURL
```

URLで指定されたリモートリポジトリから歴史をローカルに持ってきて、ワーキングツリーを展開する

GitHubのappi-githubというアカウントのclone-sampleをクローンしたい場合

HTTPS
```
$ git clone https://github.com/appi-github/clone-sample.git
```

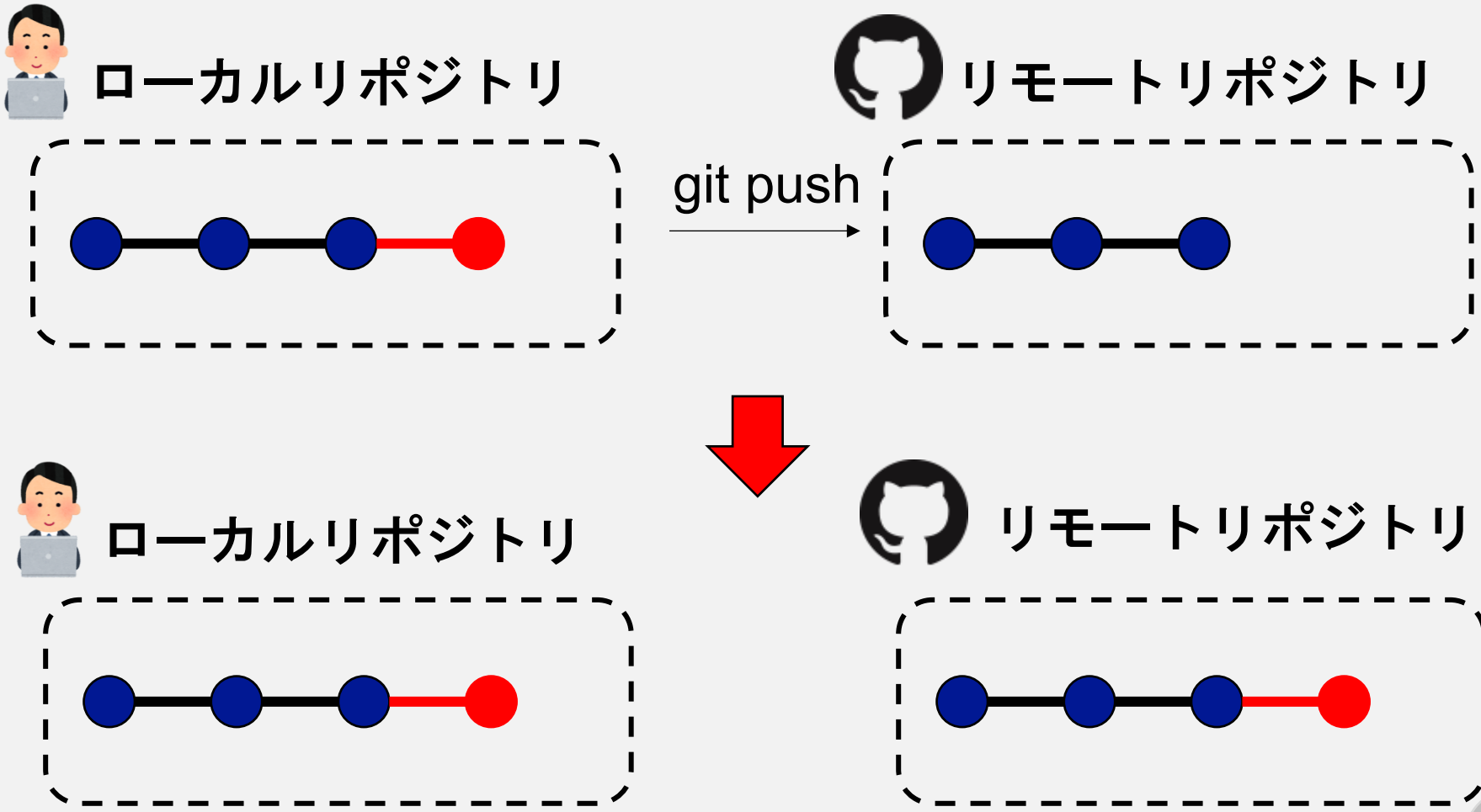
SSH
```
$ git clone git@github.com:appi-github/clone-sample.git
```

アカウント名　　リポジトリ名

カレントディレクトリにclone-sampleというディレクトリが作成され、そこにワーキングツリーが展開される

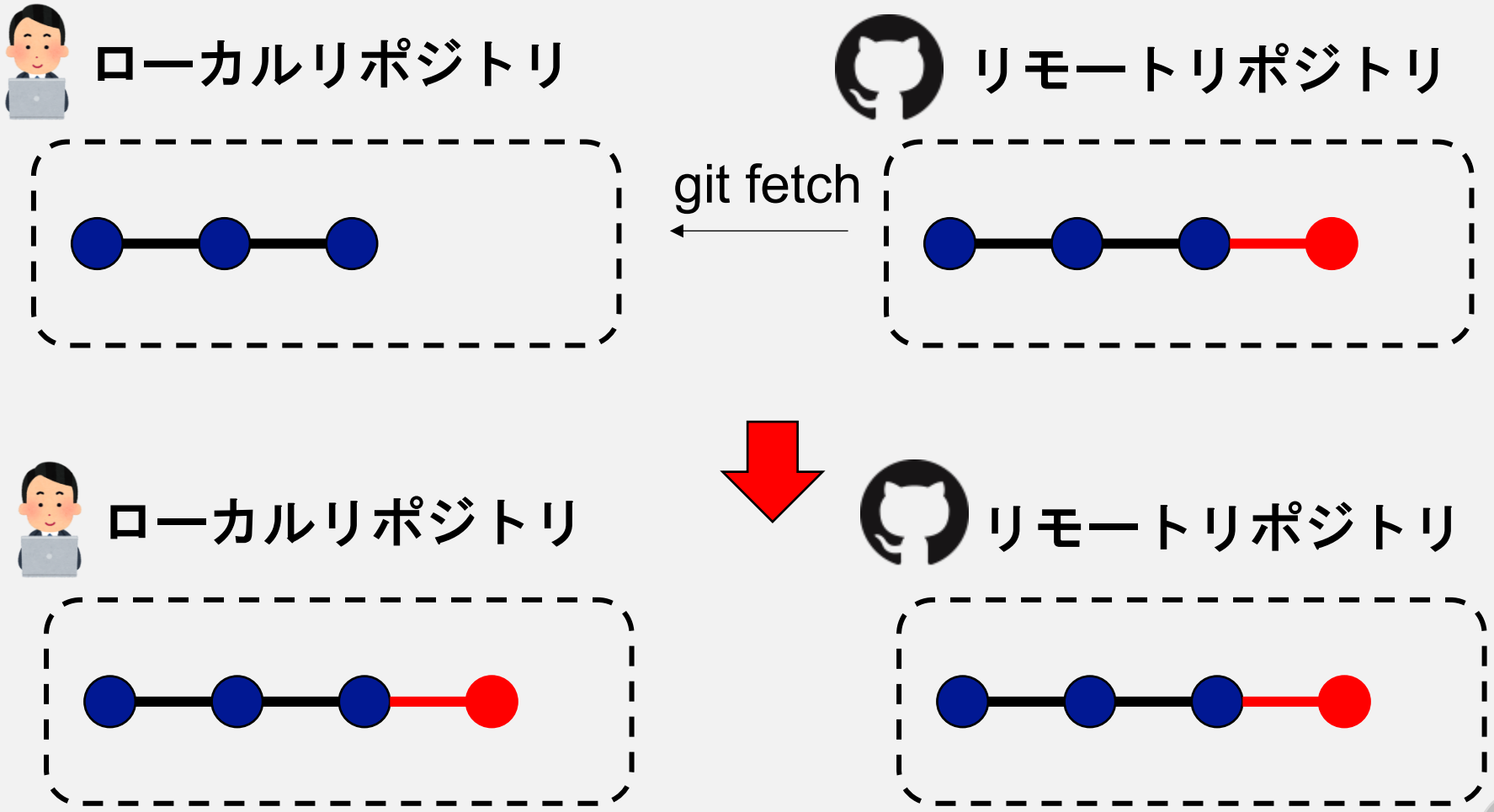
プッシュ

ローカルで変更された「歴史」をリモートに反映させる



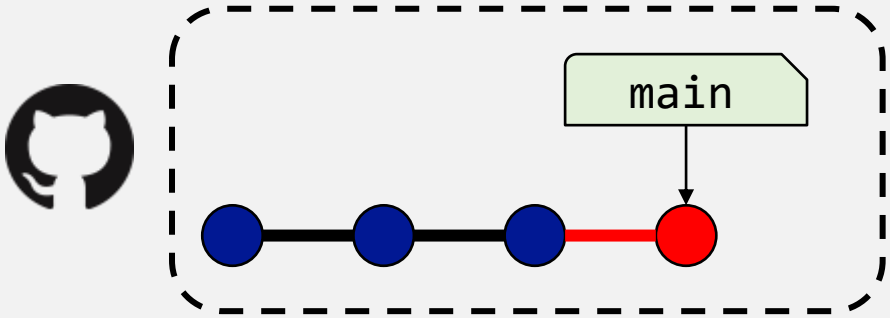
フェッチ

リモートで変更された「歴史」をローカルに取ってくる



フェッチ

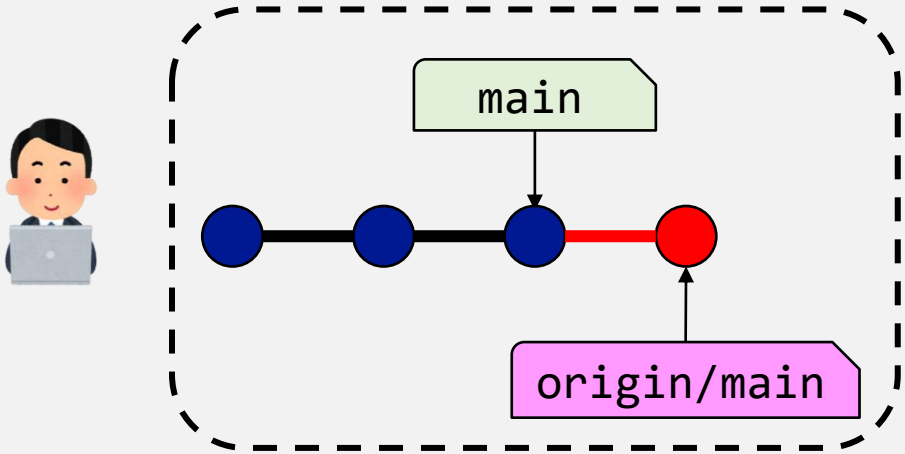
リモートリポジトリ



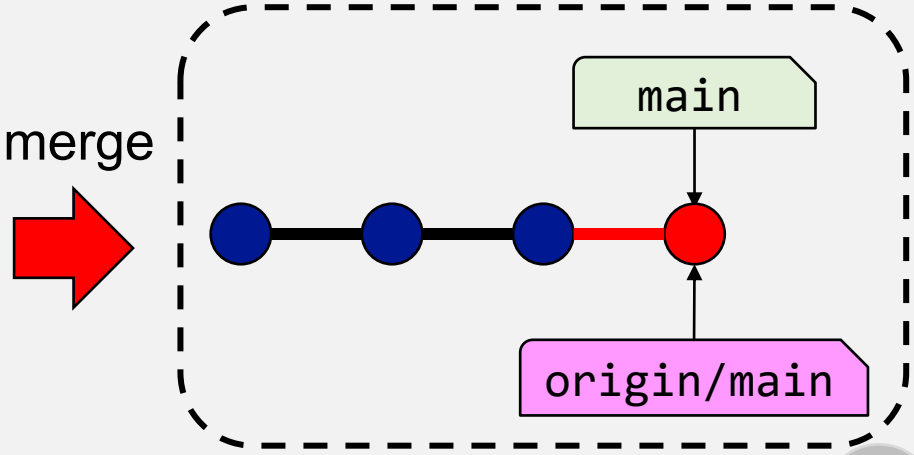
フェッチは歴史を取ってくるだけ
リモートのmainは「origin/main」
マージすることで修正を取り込む

fetch

ローカルリポジトリ



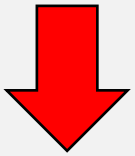
merge



上流ブランチとリモート追跡ブランチ

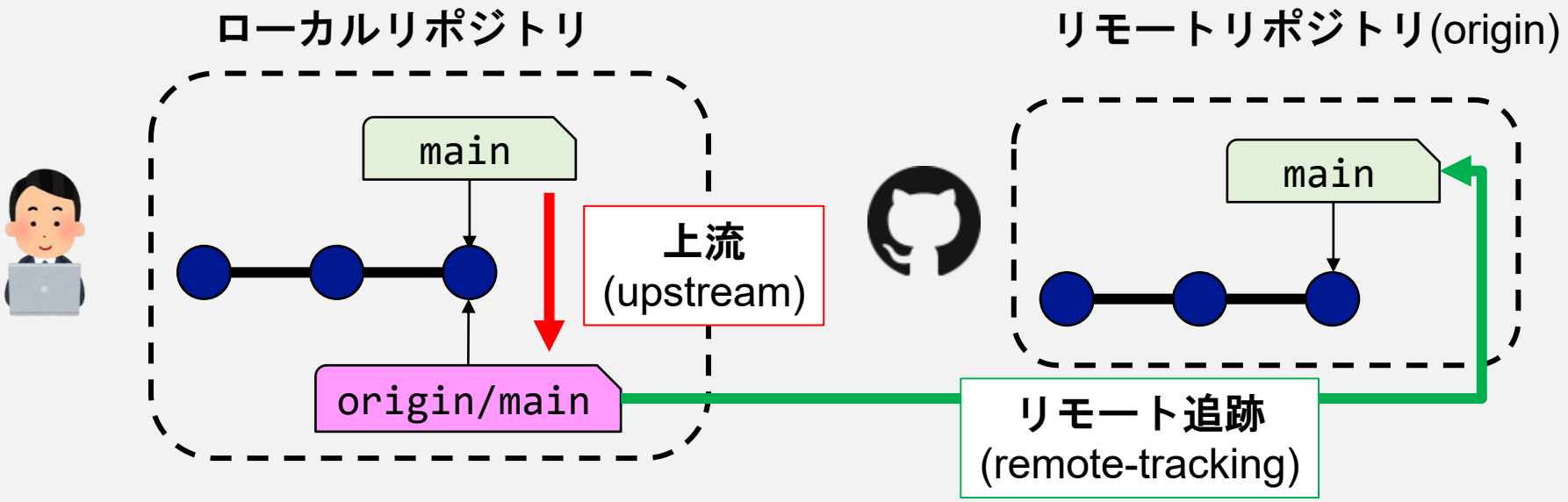
リモートリポジトリ

- Gitでは、全てのリポジトリが全ての履歴を持つ
- リモートリポジトリにアクセスして、歴史の「同期」を取る必要がある



「上流ブランチ」と「リモート追跡ブランチ」を使う

上流ブランチとリモート追跡ブランチ

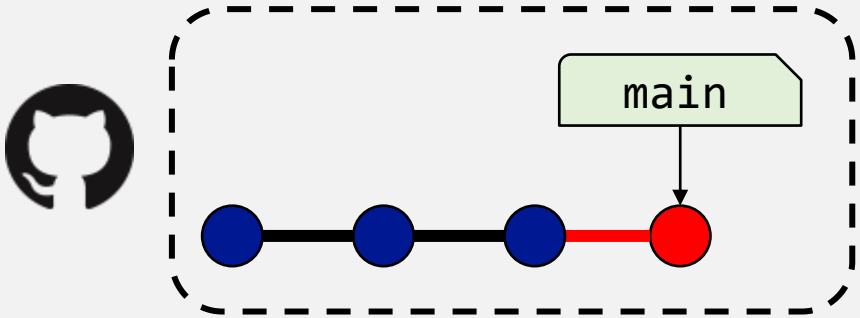


ローカルのmainの上流ブランチがローカルのorigin/main
ローカルのorigin/mainは、リモート(origin)のmainを追跡

上流ブランチはfetch, merge, rebaseでデフォルトの対象ブランチとなる

上流ブランチとリモート追跡ブランチ

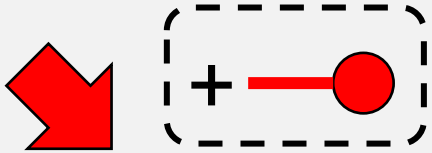
リモートリポジトリ(origin)



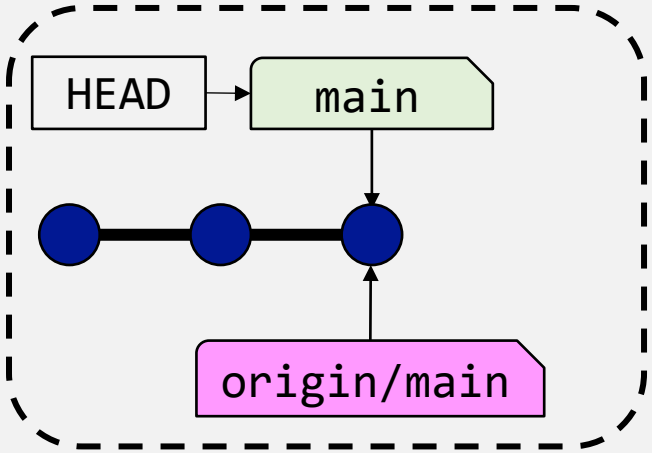

```
$ git fetch
```

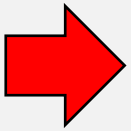
＝

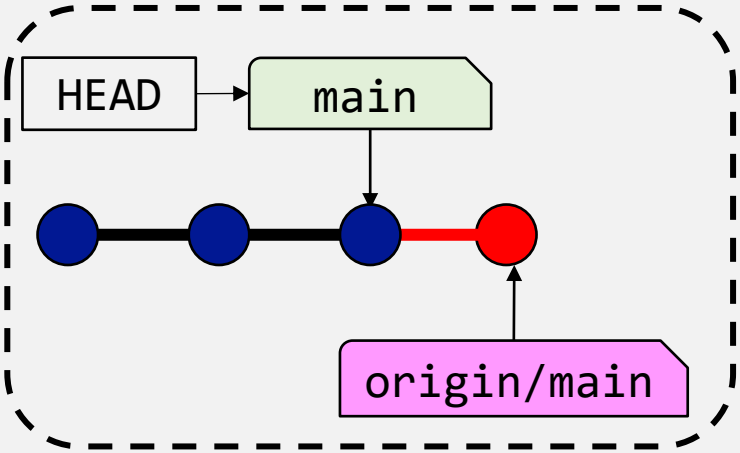
```
$ git fetch origin main
```



ローカルリポジトリ







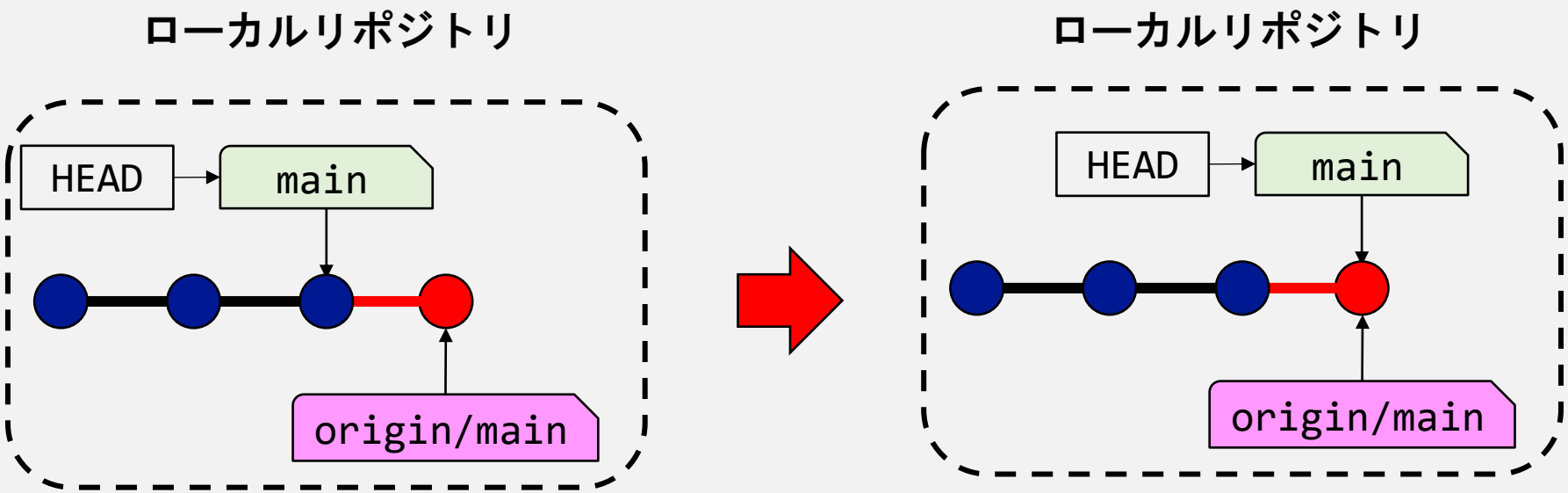
origin/mainは動くが、mainは動かない

上流ブランチとリモート追跡ブランチ

```
$ git merge
```

＝

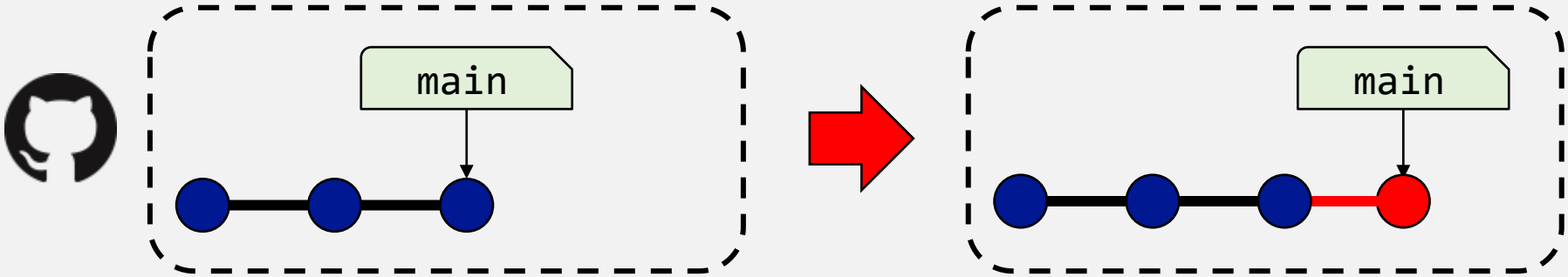
```
$ git merge origin/main
```



マージして初めてローカルのmainが動く
マージのデフォルト対象は上流ブランチ

上流ブランチとリモート追跡ブランチ

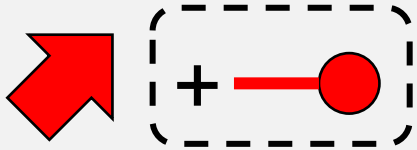
リモートリポジトリ(origin)



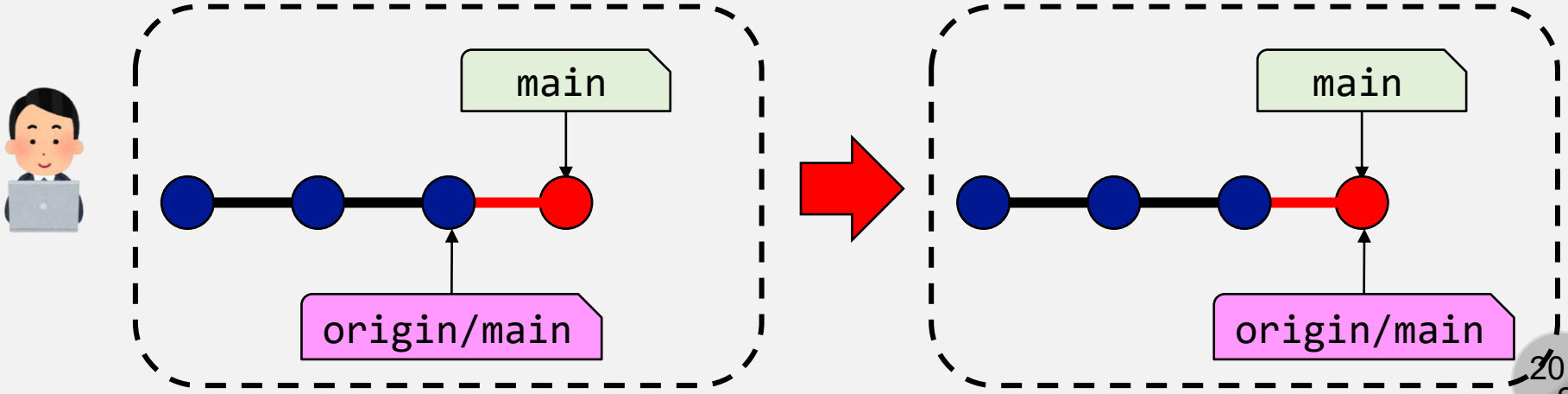

```
$ git push
```

＝

```
$ git push origin main
```



ローカルリポジトリ



上流ブランチとリモート追跡ブランチ

動くもの、動かないもの

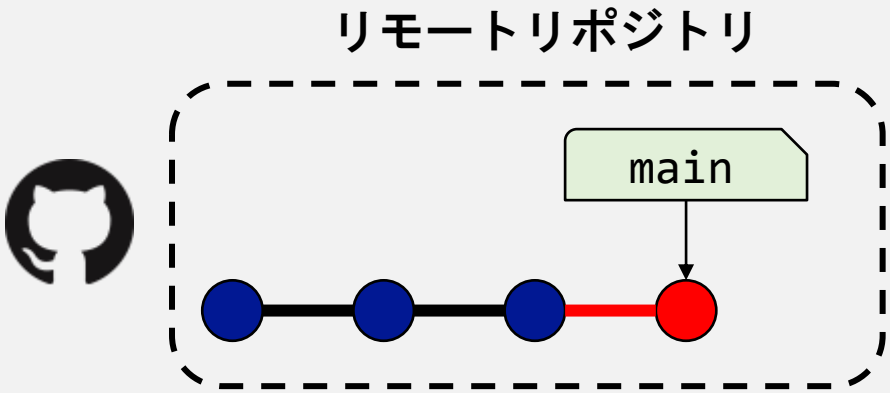
git fetch、git pushによりローカルのブランチは動かない
動くのはローカルにある上流ブランチ
git merge, rebaseではローカルのブランチのみ動く

同期するもの

`git fetch:` リモートのmainとローカルのorigin/mainが同じ状態に
`git merge:` ローカルのmainとローカルのorigin/mainが同じ状態に
`git push :` ローカルのmain,origin/main、リモートのmainが同じ状態に

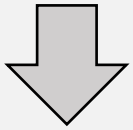
どのコマンドで何が動くか、何が同期されるのかを把握すること

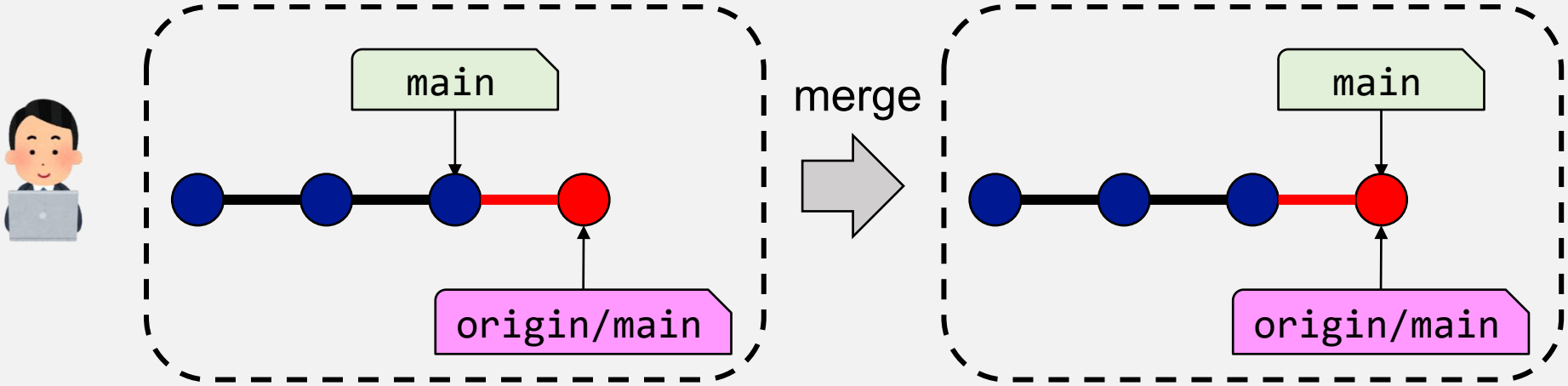
git pull



フェッチとマージを続けて行う

```
$ git pull
```





事故が起こりやすいので、慣れるまでは使わないこと

git remote

リモートリポジトリの設定、変更、削除、表示を行う

```
$ git remote -v
```

リモートリポジトリのURLを表示

実行例→

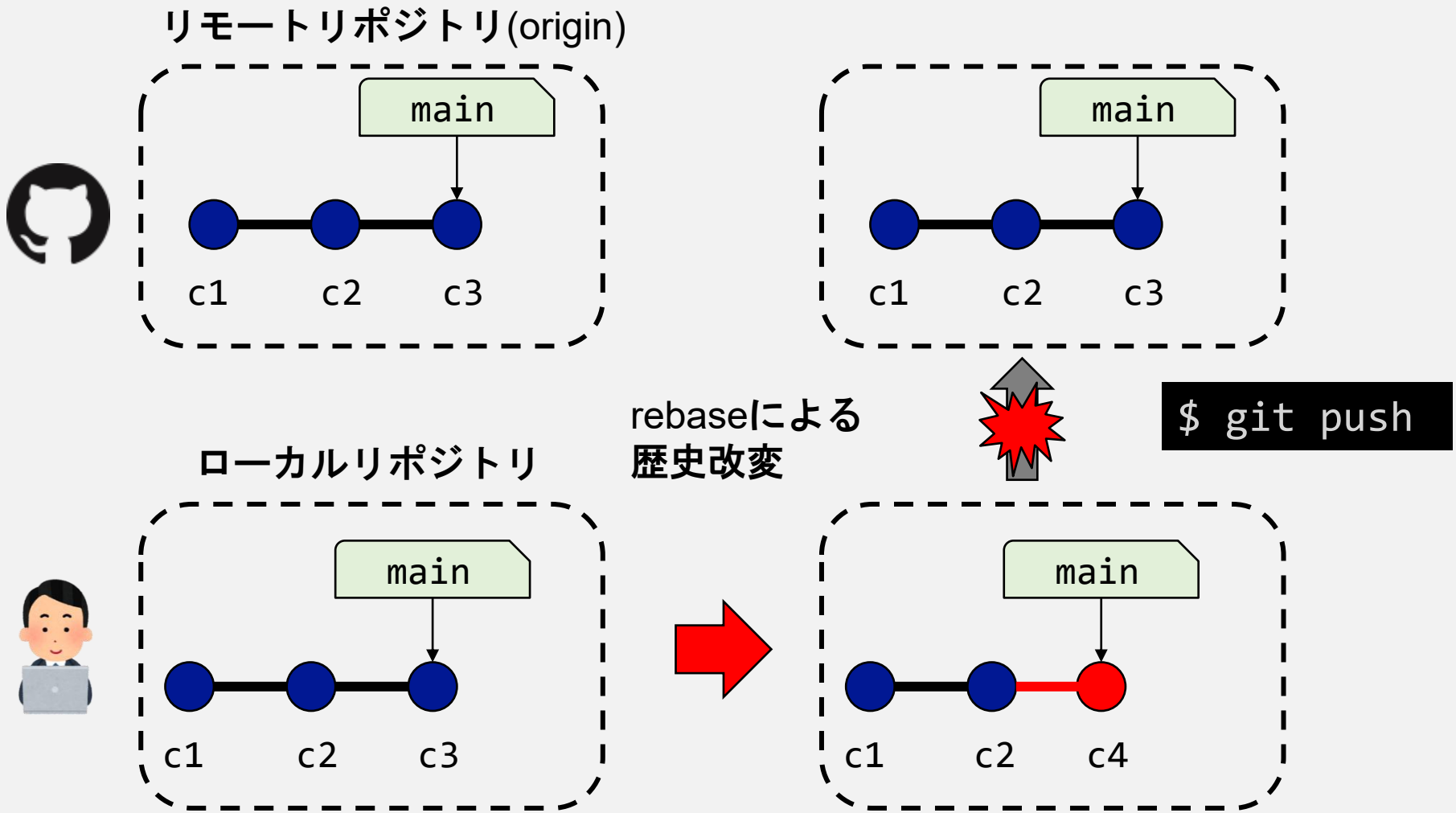
```
origin  https://github.com/appi-github/clone-sample.git (fetch)
origin  https://github.com/appi-github/clone-sample.git (push)
```

```
$ git remote add 名前 URL
```

リモートリポジトリのURLに名前(通常はorigin)を付けて登録
GitHubにローカルリポジトリを登録する際に必要
その後、上流ブランチを設定しながらプッシュする

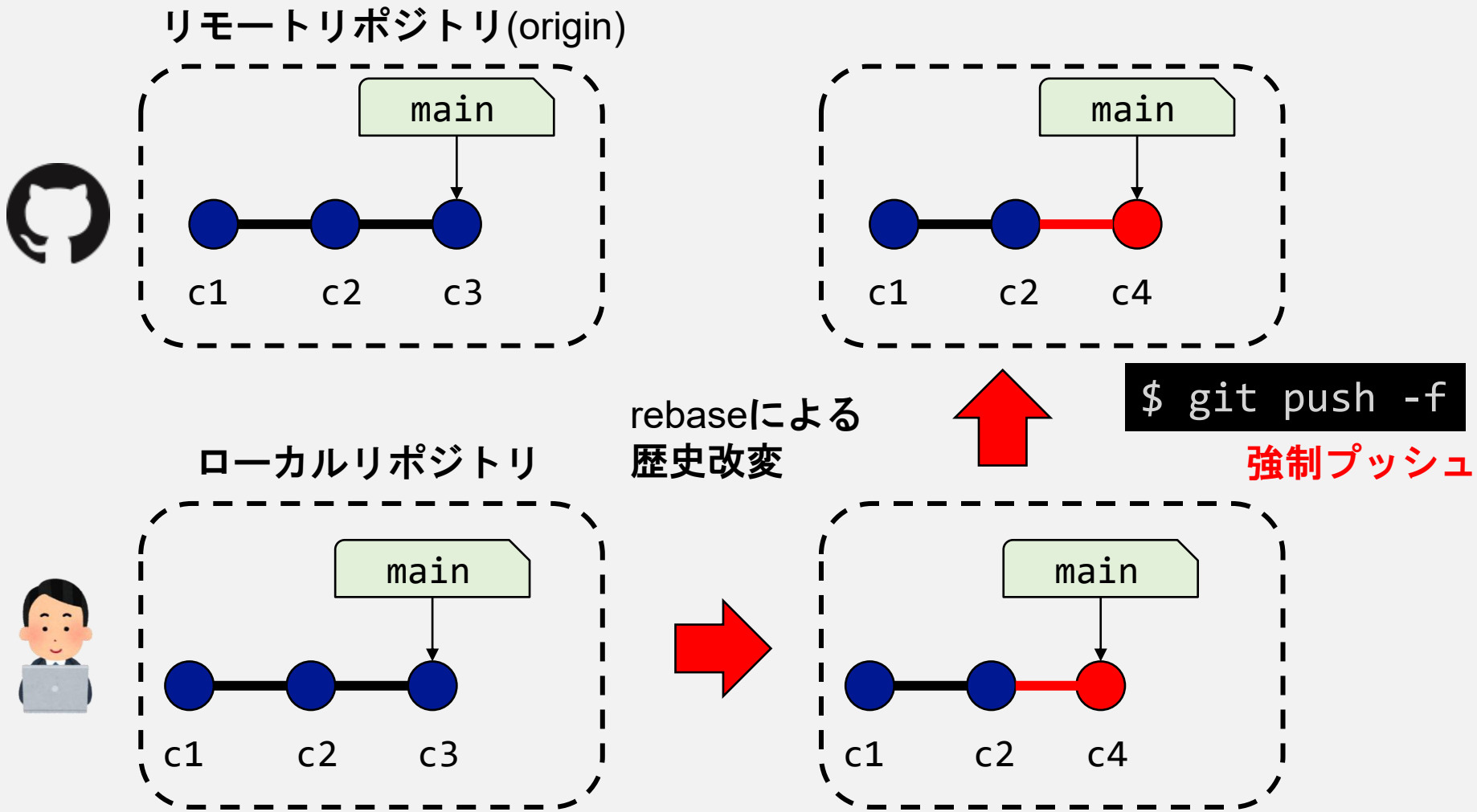
```
$ git push -u origin main
```

プッシュしたブランチのリベース



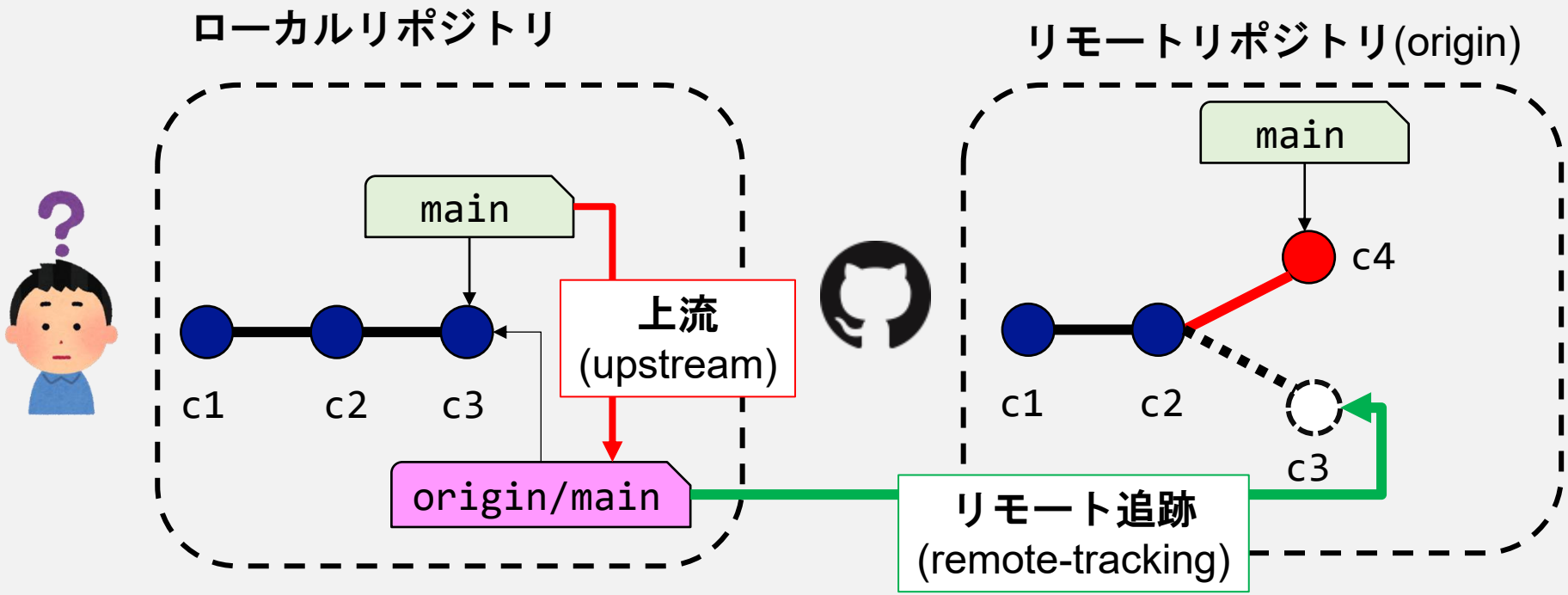
リベースにより歴史がリモートと矛盾するとプッシュできなくなる

プッシュしたブランチのリベース



強制プッシュ(-f)により、歴史を上書きできる

プッシュしたブランチのリベース



改変された歴史が強制プッシュされると、他の人のリポジトリと歴史が矛盾する

リモートにプッシュ済みのブランチをリベースしない

まとめ

Gitにおける作業の流れ

1. git fetch によりリモートの更新をダウンロード
2. git mergeによりリモートの更新を取り込む
3. git switch -c newbranchにより新しいブランチを作成して作業開始
4. 作業が終了したらmainにマージ
5. git pushにより、ローカルの更新をアップロード

Gitのリモートとローカル

- どのコマンドでどのブランチが動くかは慣れるまで把握しづらい
- 「上流ブランチ」と「リモート追跡ブランチ」の挙動に注目して理解すると良い